AI Chat Memory, Explained

> **Audience:** a new AI Engineer on this team who has used an LLM API but has never built a memory system.
> **Promise:** by the end you will know what our four memory types are, when each one is read, when each one is written, why a write can be *refused*, and how we prove any of it works.
>
> Every output block below is **real** — produced by `scripts/explain_chat_memory.py`, which calls the actual policy and gateway code with in-process fakes (no DB, no API key, no network).

```
python scripts/explain_chat_memory.py
```

---

0. The problem memory solves

An LLM API call is **stateless**. The model has no idea what you said 30 seconds ago. The only thing it ever sees is the single blob of text you send it.

So "memory" in an AI product is never magic inside the model. It is always the same boring loop:

```
user message ──► DECIDE what to look up ──► FETCH it ──► PASTE it into the prompt ──► model answers
                                                                                          │
                                                                                          └──► maybe SAVE something for next time
```

Everything in this document is one of those four verbs: **decide, fetch, paste, save**.

The hard part is not fetching. The hard part is **restraint** — not pasting the wrong user's data, not pasting a fact the user retracted, and not saving something the user never asked you to remember. That is why most of our code is *policy*, not *storage*.

---

1. The four memory types

We split memory by **how long it lives and who is allowed to write it**. This is the single most important table in the document.

Type	Human analogy	Lives for	Who writes it	Where it lives
`short_term`	What you're holding in your head right now	The session (20 turns / 30 min idle)	The system, every turn, automatically	RAM (`session_buffer.py`)
`long_term`	Your colleague knowing you prefer Vietnamese and short answers	Until the user changes or deletes it	**Only the user**, explicitly	Postgres `chat_profiles`
`episodic`	"That passport task we agreed on last week"	Until done, expired, or deleted	System proposes, **user approves**	Postgres `task_episodes`
`semantic`	The company handbook on the shelf	Owned by the company, not the chat	Nobody, from chat. **Read-only.**	Vector store / company RAG

The enum lives in `_chat_contracts_common.py:34`:

```
class MemoryType(StrEnum):
    SHORT_TERM = "short_term"
    LONG_TERM = "long_term"
    EPISODIC = "episodic"
    SEMANTIC = "semantic"
```

Beginner trap: most tutorials call "stuff the last 10 messages into the prompt" *memory*. That is only our `short_term` row. The other three rows are what make an assistant useful across days instead of across minutes — and they're where all the safety questions live.

---

2. Scope and namespace — the anti-leak primitive

Before anything is read or written, we build a **namespace**: a compound key that pins the operation to one tenant, one user, one session, one feature, one memory type, one record. If any part of it disagrees with the request, the operation is refused rather than served.

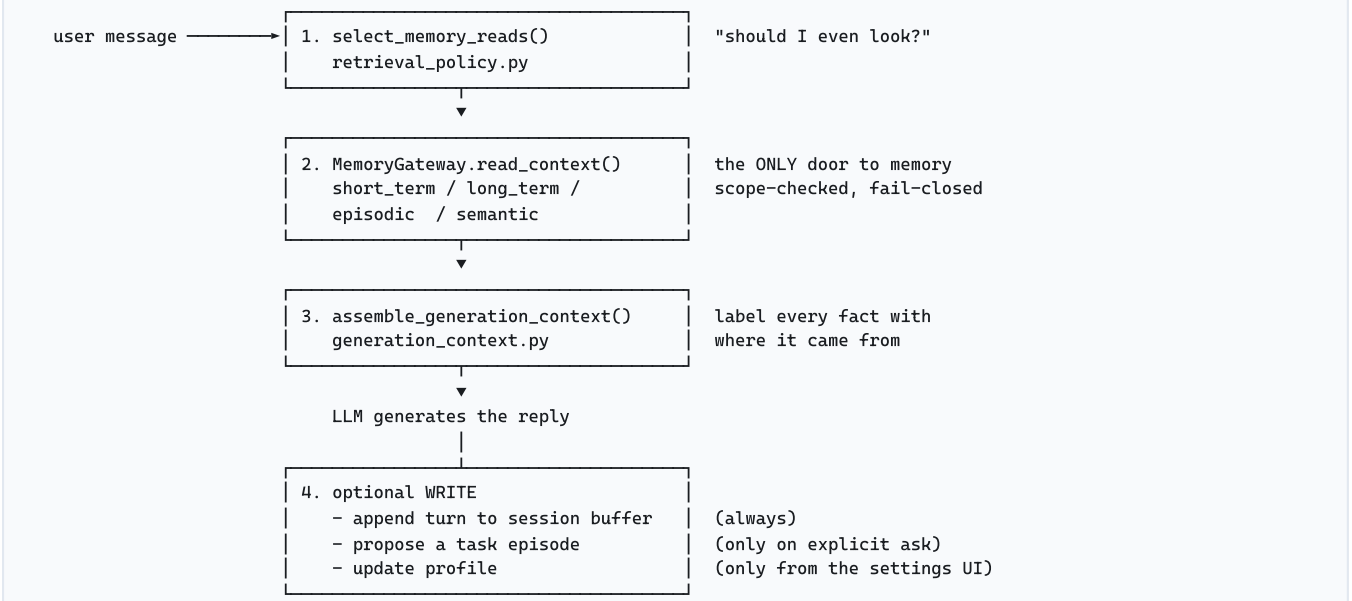
```
======================================================================
1. MemoryNamespace.logical_key() -- one key per (user, session, type, record)
======================================================================
u_thanh/s_2026_08_25/ai_chat/short_term/s_2026_08_25
u_thanh/s_2026_08_25/ai_chat/long_term/profile
u_thanh/s_2026_08_25/ai_chat/episodic/ep_9f2c
u_thanh/s_2026_08_25/ai_chat/semantic/chunk_17
```

Read that as a filing-cabinet address: *user → session → feature → drawer → folder*.

Why bother? Because the #1 catastrophic bug in a memory system is **cross-user leakage** — user A's chat quietly retrieving user B's data. Passing a `user_id` around as a loose string makes that bug one typo away. Making the key a value object means the guard is mechanical, and it fires in exactly one place (`memory_gateway.py`).

---

3. Life of one chat turn



Sections 4–8 walk each box with real output.

4. Step 1 — Decide: does this message deserve a lookup?

Naive systems search every store on every turn. That is slow, expensive, and *actively harmful*: retrieval returns something plausible-looking for any query, so searching when the user didn't ask for it is how you get confident nonsense.

So `retrieval_policy.py` is a **deterministic gate** — plain token matching, no LLM call, no embedding:

```
======================================================================
2. select_memory_reads() -- which stores does THIS message unlock?
======================================================================
'Hạn chót của việc đó là thứ mấy?'
    short_term=True long_term=True episodic=off semantic=off
'What does our company policy say about remote work?'
    short_term=True long_term=True episodic=off semantic=QUERY('What does our company policy say about remote work?')
'Tôi còn tác vụ trước nào đang mở không?'
    short_term=True long_term=True episodic=off semantic=off
'Create a task to renew the Da Nang ID cards'
    short_term=True long_term=True episodic=QUERY('Create renew Da Nang ID cards') semantic=off
'Đừng tạo tác vụ cho việc này nhé'
    short_term=True long_term=True episodic=off semantic=off
```

Four things to notice:

1. `short_term` **and** `long_term` **are always on.** They're cheap (RAM + one indexed row) and always relevant. Only the two *search* stores are gated.
2. **"company policy" flipped semantic on.** The phrase `company policy` is a cue in `_SEMANTIC_CUES`. No cue, no company-RAG search.
3. **A task-creation request also opens episodic.** Counter-intuitive, but necessary: to know whether this new task *replaces* an existing one, the write turn must first see the tasks it might be replacing.

4. `"Đừng tạo tác vụ..."` **("don't create a task") stays off.** Negation markers are checked before the directive verbs, so the word `tạo` ("create") in a refusal doesn't trigger a write.

4b. The query is not the question

When we do search episodes, we don't search with the raw sentence:

```
======================================================================
2b. episodic_search_text() -- frame words stripped, only the WHICH survives
======================================================================
'Tôi còn tác vụ trước nào đang mở không?'
    -> ''
'What was my previous task about the Hai Phong passports?'
    -> 'Hai Phong passports'
```

Why this exists — and it's the single best lesson in this codebase.

Our Postgres search predicate is `search_vector @@ plainto_tsquery('simple', ...)`, and `plainto_tsquery` **ANDs every token**. Search with the whole question and the row must contain "*tôi*" AND "*còn*" AND "*nào*" AND "*không*" — which no stored task title ever does. The store was healthy, the writes were landing, and the system still reported itself as amnesiac. The bug was in the query, not the memory.

So the cue decides **whether** to search; the content words decide **what for**.

And when nothing meaningful survives (first line above → `''`), the correct answer is *not* to search on leftover filler. "Do I have any open tasks?" is a request to **enumerate** (`list_task_episodes`), not to search. Searching and enumerating are different questions, and conflating them made a working store look broken.

---

5. Step 2 — Short-term: a bounded buffer, not a transcript

```
======================================================================
3. InMemoryChatSessionBuffer -- newest-N with an inactivity TTL
======================================================================
after append #1: kept=['t1']
after append #2: kept=['t1', 't2']
after append #3: kept=['t1', 't2', 't3']
after append #4: kept=['t2', 't3', 't4']
after 1801s idle (ttl=1800): kept=[]  <- session forgotten
```

(The demo uses `max_turns=3` so the eviction is visible in four lines. Production defaults are `CHAT_MEMORY_MAX_TURNS=20` and `CHAT_MEMORY_TTL_SECONDS=1800`.)

Two bounds, two different jobs:

- `max_turns` bounds cost. Context windows are finite and priced per token; an unbounded buffer is an unbounded bill and, eventually, a hard API error.
- `ttl_seconds` bounds *staleness and privacy*. It is an **inactivity** TTL, refreshed on every append — so an active conversation never expires mid-flow, but a session abandoned for 30 minutes is dropped rather than resurrected hours later with stale assumptions.

It's in-process RAM on purpose: this data is worthless after the session, so persisting it would create a privacy liability with no product benefit.

---

6. Step 3 — The Gateway: one door, fail closed

Every memory read and write goes through `MemoryGateway`. No feature code touches a repository directly. One door means one place to enforce scope, one place to emit telemetry, one place to review when something leaks.

Here it answers "*How's that earlier Cần Thơ passport task going?*" against a store holding three episodes:

id	title	status	note
`ep_old`	Submit Cần Thơ passport docs on 5 Sep	`user_approved`	superseded by `ep_new`
`ep_new`	Move Cần Thơ submission to 12 Sep	`user_approved`	`supersedes="ep_old"`
`ep_draft`	Renew Hải Phòng ID cards	`system_generated`	proposed, never approved

```
======================================================================
4. MemoryGateway.read_context() -- one fail-closed read across four stores
======================================================================
turns            : ['Hạn chót là thứ Ba.', 'Đính chính: dời sang thứ Tư.']
profile          : vi / trợ lý biệt danh Hải Âu / ngắn gọn
episodes returned: ['ep_new']
   ep_draft dropped (system_generated -> retrieval_eligible=False)
   ep_old  dropped (superseded by ep_new)
degraded         : False  sources=[]

observability events (metadata only, no user text):
  {"feature": "ai_chat", "memory_type": "short_term", "operation": "read", "outcome": "success", "result_count": 2,
"filtered_count": 0, "latency_ms": 0, "reason_code": null}
  {"feature": "ai_chat", "memory_type": "long_term", "operation": "read", "outcome": "success", "result_count": 1,
"filtered_count": 0, "latency_ms": 0, "reason_code": null}
  {"feature": "ai_chat", "memory_type": "episodic", "operation": "read", "outcome": "requested", "result_count": 0,
"filtered_count": 0, "latency_ms": 0, "reason_code": null}
  {"feature": "ai_chat", "memory_type": "episodic", "operation": "read", "outcome": "success", "result_count": 1,
"filtered_count": 2, "latency_ms": 0, "reason_code": null}
```

Three episodes stored, one returned. Both drops are safety features:

- `ep_draft` is `system_generated` — the model proposed it, the user never approved it. Feeding an unapproved draft back as fact is how an assistant "remembers" something that never happened. `retrieval_eligible` is a *generated column* in Postgres (`validation_status IN ('user_approved','completed')`), so no code path can set it by hand.
- `ep_old` was explicitly replaced. Returning both dates would let the model answer "5 September" — the classic **stale answer**, which in a scheduling product is worse than saying nothing.

Read the telemetry line too. `result_count: 1, filtered_count: 2` tells you the store had data and policy removed it. Without that split, "we returned 1 result" and "the store is nearly empty" look identical in a dashboard — and you'd debug the wrong layer. Note what the event does *not* contain: no message text, no titles, no user content. Logs get shipped to third parties; making them structurally incapable of carrying subject data is cheaper than auditing them later.

6b. Degrade vs deny

Two failure modes that must behave differently:

```
======================================================================
7. Degrade vs deny -- a missing source is survivable, a wrong scope is not
======================================================================
semantic asked for, adapter absent -> degraded=True sources=['semantic']
turn still answered with: turns=2 profile=True
  {"feature": "ai_chat", "memory_type": "semantic", "operation": "read", "outcome": "requested", ...}
  {"feature": "ai_chat", "memory_type": "semantic", "operation": "read", "outcome": "degraded", ..., "reason_code":
"not_configured"}
reading another user's scope -> NamespaceAccessDenied: requested scope does not match the verified chat scope
  {"feature": "ai_chat", "memory_type": "short_term", "operation": "read", "outcome": "denied", ..., "reason_code":
"scope_denied"}
```

- **Company RAG is down → degrade.** The turn still happens with the memory we *do* have, and `degraded_sources` records the gap so the UI can say "I couldn't check the handbook" instead of silently answering from nothing.
- **Scope mismatch → deny.** No partial result, no empty list, no "best effort". It raises, and it emits `outcome: denied` with `reason_code: scope_denied` — which is a page-worthy alert, not an info log.

"Availability problem → degrade. Authorization problem → deny." Getting these backwards is how systems leak.

---

7. Step 4 — Paste: label every fact with its source

We never concatenate memory into one anonymous prompt blob. Each fact is wrapped in a **labeled section**, and the labels carry a declared precedence order:

```
======================================================================
5. assemble_generation_context() -- labeled sections + conflict precedence
======================================================================
current_instruction : current_instruction = 'Tác vụ trước về hộ chiếu Cần Thơ thế nào rồi?'
active_session_turns: 2 turns
stored_preference   : stored_preference (advisory=False)
advisory_episodes   : ['Dời ngày nộp hồ sơ hộ chiếu Cần Thơ sang 12/9'] (advisory=True)
company evidence    : None
conflict_precedence : ['current_instruction', 'current_company_evidence', 'stored_preference', 'advisory_episode']
response_mode       : normal
```

Precedence, highest first:

1. `current_instruction` — what the user just typed always wins. If they say "actually, Thursday", no stored memory may override it.
2. `current_company_evidence` — the handbook outranks anything the assistant inferred.
3. `stored_preference` — the user's own explicit settings.
4. `advisory_episode` — lowest, and flagged `advisory=True`.

`advisory=True` is the load-bearing flag. An episode is a *record of a past proposal*, not a fact about the world. It says "you and I agreed to move this to 12 September", not "the submission is on 12 September". The distinction survives all the way into the prompt so the model can hedge appropriately rather than assert.

---

8. Writes — the part that fails closed

Reads decide what the model sees. **Writes decide what the product believes tomorrow**, so every write path is guarded by a pure policy function that runs before any adapter is reached.

8a. Long-term profile: explicit user acts only

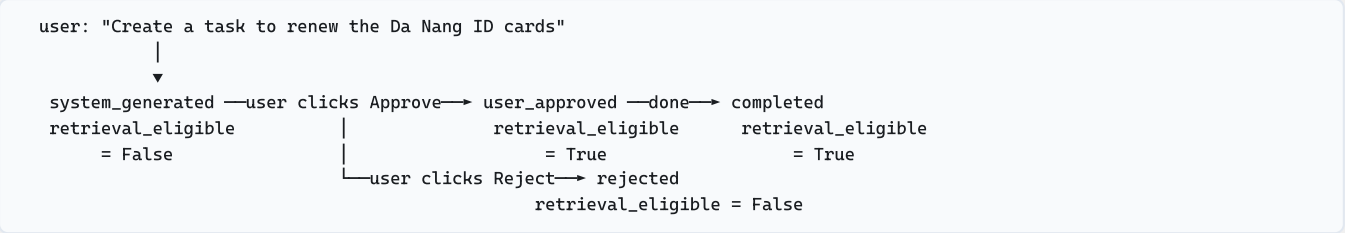
```
======================================================================
6. Write policies -- fail closed on anything but an explicit user act
======================================================================
provenance=explicit_user_config         -> ACCEPTED
provenance=system_generated_chat_task   -> REJECTED: long-term writes require explicit_user_config provenance
persona of 201 chars (cap 200)           -> REJECTED: assistant_persona exceeds the compact profile bound
```

`profile_policy.py` accepts a profile write only with `EXPLICIT_USER_CONFIG` provenance — the settings UI, a literal "remember this" request, or a trusted admin config. **The model can never write to long-term memory by inferring a preference.**

That's a deliberate product decision, not a limitation. A system that silently learns "you seem to prefer English" from one English message becomes unpredictable and un-debuggable, and the user has no mental model of why it changed. Our long-term memory holds exactly four fields — `language`, `timezone`, `assistant_persona`, `response_tone` — each capped at 200 characters. Small, inspectable, user-owned.

8b. Episodes: proposed by the system, promoted by the user

An episode's whole life is its `validation_status`:



A proposal is stored but **invisible to retrieval** until a human promotes it. This is human-in-the-loop applied to memory: the model may draft, only the user may commit.

Some hard-won details:

- **The server owns identity.** The model supplies only `task_title`, a paraphrase, an action plan, missing-information notes, and citations. `episode_id`, `record_id`, `validation_status`, timestamps and scope are all assigned server-side in `_new_task_episode`, from verified session metadata. A model that hallucinated an `episode_id` could otherwise overwrite a different task.
- **`record_id` is `sha256(user_id ␟ session_id ␟ turn_id)`.** Deterministic per turn, so a retried request updates the same row instead of creating a duplicate task.
- **Bounds are enforced twice** — in the Python dataclass *and* as Postgres `CHECK` constraints (title ≤ 200, paraphrase ≤ 1000, ≤ 20 plan items, etc.). The DB is the last line of defence against a model returning a 40 KB "action plan".
- **The schema has no slot for raw content.** No email bodies, no attachments, no transcripts, no RAG chunk text — only derived task metadata. You cannot leak a field that does not exist.

8c. Where it actually lands

```
-- src/cowork_agent/persistence/migrations/004_task_episodes.sql
retrieval_eligible boolean GENERATED ALWAYS AS (
    validation_status IN ('user_approved', 'completed')
) STORED,
search_vector tsvector GENERATED ALWAYS AS (
      setweight(to_tsvector('simple', task_title), 'A')
   || setweight(to_tsvector('simple', minimal_request_paraphrase), 'B')
   || setweight(to_tsvector('simple', action_plan::text), 'C')
   || setweight(to_tsvector('simple', missing_information::text), 'C')
) STORED,
PRIMARY KEY (tenant_id, user_id, feature, chat_session_id, record_id),
```

Two ideas worth stealing:

- **Generated columns for invariants.** `retrieval_eligible` is *derived*, so "an unapproved episode is retrievable" is not a bug we can write — it's a state the database cannot represent.
- **`'simple'`, not `'english'`.** Postgres ships no Vietnamese text-search configuration, and the English stemmer would mangle Vietnamese tokens. `'simple'` does no stemming and has no stopword list — which is precisely why the stopword filtering in §4b has to be an explicit list in Python.

8d. Deletion and retention

- `delete_all_memory()` clears this user's profile + episodes + session buffer, and returns a count. It **never** touches company RAG — that's org-owned, not user-owned.
- `MemoryPurgeCoordinator.purge_expired(now)` sweeps rows past `expires_at`, driven by optional `CHAT_PROFILE_RETENTION_SECONDS` / `CHAT_EPISODE_RETENTION_SECONDS`.
- Deletions emit the same telemetry events as reads, so "user asked to be forgotten" is auditable.

---

9. How we know any of this works

Code review can't tell you whether an assistant *remembers*. Only an eval can. Ours lives in `features/ai_chat/memory_eval/`; results are recorded in `MODEL-MEMORY-EVAL-LEADERBOARD.md`. (The neighbouring `memory-evals.md` documents an external system, *Waku* — useful for comparison, not a description of ours.)

9a. Pass/fail is not enough

`scoring.py` grades each answer into one of five outcomes, because *how* you fail matters more than *that* you failed:

Outcome	Meaning	How bad
`PASS`	Recalled the fact, **or** correctly declined an unseeded question	Correct
`MISS`	Didn't recall it, but asserted nothing false	Honest gap
`STALE`	Asserted a **superseded** fact ("5 September" after it moved to 12)	Bad
`INVENTED`	Asserted a fact never given — a hallucination	Worst
`NO_ANSWER`	The provider returned nothing at all	Not a grade — the *absence* of one

`NO_ANSWER` is worth pausing on. A provider outage is not a memory failure, and folding the two together once produced three confident conclusions from what turned out to be a brief outage. If your harness can't distinguish "the system forgot" from "the system never answered", your numbers are fiction.

9b. Three arms per probe

The same question is asked three ways (`arms.py`):

- `full` — memory readable. Can it recall?
- `ablated` — one scope masked off at the gateway. Does it now correctly *refuse*?
- `control` — never seeded at all. A pass here means the model knew the answer from pre-training; memory earned nothing.

The ablation is done by subclassing `MemoryGateway` and forcing one scope's read off — using the *same* disabled-read objects `retrieval_policy` builds when no cue fires, so a masked arm is indistinguishable from a genuine no-cue turn. An arm is a statement about what can be **read**, never about what may be stored.

9c. Three outcomes → one verdict

`verdicts.py` collapses the arms into a verdict, and the scoreboard sorts **worst first**, so the interesting rows are never buried under a wall of passes:

```
unreadable > dangerous > broken > leaked > scope_did_nothing > scope_earned_it > restraint_held
```

- `unreadable` ranks above real failures because the run itself failed for that question — and a run that failed cannot support a claim about the product.
- `dangerous` = `STALE` or `INVENTED` on any arm.
- `leaked` = the control arm passed a recall probe. It invalidates the matching `full` pass.
- `scope_earned_it` = recalled with memory, refused without it, refused unseeded. This is the *only* verdict that proves your memory system — rather than the model's world knowledge — did the work.
- `restraint_held` sorts last on purpose: it's the behaviour we wanted, and never a finding.

The control arm is the one beginners skip, and it's the one that keeps you honest. Real numbers from the current leaderboard: the top models reach a 90% overall pass rate but only **45–50% Earned-It**. That gap is the value of the control arm, stated in one number.

---

10. Beginner FAQ

Why not just send the whole conversation every turn? Cost grows quadratically, latency grows with it, and past ~20 turns models start losing things in the middle. Bounded buffer + targeted retrieval is cheaper *and* more accurate.

Why not let the LLM decide when to search? Non-deterministic, costs an extra call, and can't be unit-tested. Our gate is exact token matching, so a rule change is a diff you can review and a test you can pin. The tradeoff is real: an unlisted phrasing simply doesn't fire. We accept that, because a *silent extra* retrieval is far more dangerous than a missing one.

Why can't the model save preferences it infers? Because the user can't see, predict, or correct an inference. Explicit-only memory is always explainable: every stored preference maps to something the user did.

Why store an approved plan instead of the raw conversation? The raw conversation is long, contains everything sensitive that was ever pasted, and has no agreed meaning. A task episode is short, structured, bounded, and represents something a human actually signed off on.

Where do I start reading the code?

To understand…	Read
The vocabulary	`domain/_chat_contracts_memory.py`
When we look things up	`retrieval_policy.py`
The one door to memory	`memory_gateway.py`
Why a write was refused	`profile_policy.py`, `episode_policy.py`
What the model actually sees	`generation_context.py`
What the DB enforces	`migrations/004_task_episodes.sql`

---

11. The five sentences worth memorizing

1. **Memory is prompt construction**, not something the model does — decide, fetch, paste, save.
2. **Split memory by write authority**, not by storage tech: system-written, user-written, user-approved, company-owned.
3. **One gateway, scope-checked, fail-closed** — availability problems degrade, authorization problems deny.
4. **A model may propose; only a user may commit.** Unapproved and superseded records must be structurally unretrievable.
5. **Measure restraint, not just recall** — and always run a control arm, or you're benchmarking the model's pre-training instead of your memory.